Web应用开发C

成都信息工程大學
Chengdu University of Information Technology

学校：成都信息工程大学

学院：软件工程学院

专业：数据科学与大数据技术

授课教师：杨与广

邮箱：yyg@cuit.edu.cn

群名称:数科221-Web应用开发C
群　号:870648816

第七章 Flask-Web表单 回顾

- Flask-WTF　　表单、特点
- 基本流程　　配置、构建表单类、创建实例、渲染HTML、提交验证

模板

MTV

- 在Web开发中，架构模式是一种组织和管理代码的方式，它能够使我们的应用程序更易于理解、扩展和维护。MTV架构是一种常见的Web开发架构，它将应用程序分为三个主要组件：模型(Model)、模板(Template)和视图(View)。
  - ✓ 模型是MTV架构的核心组件，它负责定义应用程序的数据结构和业务逻辑
  - ✓ 模板是MTV架构的表示层组件，它负责生成应用程序的用户界面
  - ✓ 视图是MTV架构的控制层组件，它负责处理用户请求，并根据请求的类型和参数来决定如何处理请求

第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

数据库

数据库

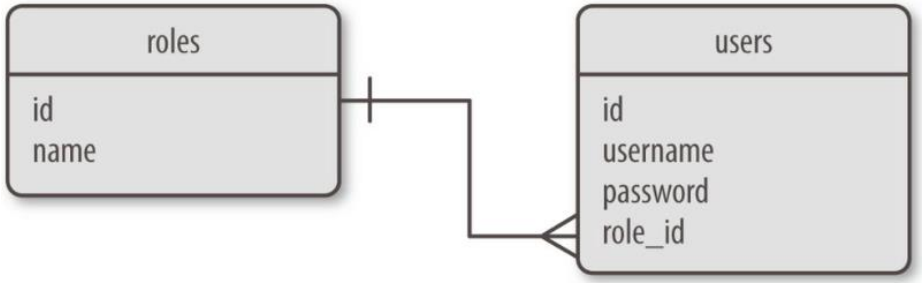
- 按照数据结构来组织、存储和管理数据的仓库
- 是一个长期存储在计算机内的、有组织的、可共享的、统一管理的大量数据的集合
- 数据库按照 一定规则保存应用的数据，应用再发起查询，取回所需的数据，实现数据的持久化应用
- Web应用最常使用基于关系模型的数据库，一般使用结构化查询语句来实现数据库的增删改查—SQL数据库
- 同时现在文档数据库和键-值对数据库也成为流行的替代选择—NoSQL数据库

数据库

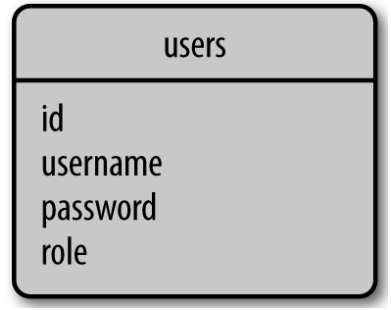
成都信息工程大學 Chengdu University of Information Technology

SQL数据库

- 关系型数据库将数据存储在表中，表就是应用中不同的实体建模
- 表中的列数固定，行数可变；列定义了表所代表实体的数据属性，行定义部分或全部的数据
- 表中有个特殊的列(**主键**)，其值代表表中各行的唯一标识
- 表中还有一个 特殊的列(**外键**)，应用同一个表或不同表中的主键，建立两个表之间的关系
- 一般使用**实体-关系图**来描述一个系统的数据结构
- 关系型数据库存储数据很高效、且避免了 重复
- 数据存储在多个表中，连接查询比较复杂
  存在一对一、一对多、多对多的关系
- 连接操作需要关系型数据库引擎提供支持



数据库



- NoSQL数据库
  - 所有不符合关系模型的数据库统称为 NoSQL 数据库
  - 一般使用集合代替表，使用文档代替记录
  - NoSQL 数据库采用的设计方式使联结变得困难，所以多数根本不支持这种操作
  - 减少了表的数量，却增加了数据重复量；修改需要更新大量文档
  - 数据重复可以提升查询速度
  - 列出用户及其角色的操作将很简单，因为无须联结
- SQL还是NoSQL数据库
  - SQL 数据库擅于用高效且紧凑的形式存储结构化数据，一般采用ACID 的范式
  - NoSQL 数据库放宽了对 ACID 的要求，从而获得性能上的优势

数据库

Python数据库框架

- 为了连接Flask 应用和数据库，我们需要数据引擎
- 大多数数据库引擎都有对应的 Python 包。Flask 并不限制你使用何种类型的数据库包，因此你可以根据自己的喜好选择使用 MySQL、Postgres、SQLite、MongoDB
- 数据框架选择时考虑的因素，选择Flask-SQLAlchemy：
  - ✓ **易用性**：若直接比较数据库引擎和数据库抽象层(ORM或ODM)，毋庸置疑选择自动实现高层的面向对象操作转换成底层的数据库指令的数据库抽象层
  - ✓ **性能**： 使用ORM或ODM将对象业务转换成数据库业务会有一定的性能消耗，但较之其所提供的的便利，这点性能消耗完全可以接受
  - ✓ **可移植性**：选择数据库时，须考虑其是否能在开发平台和生产平台中使用，SQLAlchemy ORM 支持多种数据库引擎
  - ✓ **Flask集成度**：选择集成了Flask的框架可以节省编写集成代码的时间，简化配置和操作

数据库

MySQL数据库基本命令

CREATE DATABASE 数据库名;	创建数据库
SHOW DATABASES;	查询数据库
DROP DATABASE 数据库名;	删除数据库
USE 数据库名;	使用数据库
CREATE TABLE table_name (column_name column_type);	创建数据表
DROP TABLE table_name ;	删除数据表
SELECT * FROM table_name ;	查询数据行
UPDATE table_name SET field=new-value	更新数据行
DELETE FROM table_name [WHERE Clause]	删除数据行

https://dev.mysql.com/doc/refman/8.2/en/tutorial.html

https://zhuanlan.zhihu.com/p/653233284

第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

Flask-SQLAlchemy

https://zhuanlan.zhihu.com/p/151373067

ORM

- 是一种程序技术，用于实现面向对象编程语言里不同类型系统的数据之间的转换
- 提供了概念性的、易于理解的模型化数据的方法
- ORM方法论基于三个核心原则：
  - ✓ 简单：以最基本的形式建模数据
  - ✓ 传达性：数据库结构被任何人都能理解的语言文档化
  - ✓ 精确性：基于数据模型创建正确标准化的结构
- 是在数据库引擎之上又进行了一层封装，对于数据的操作，无需再去编写原生sql，取代代之的是基于面向对象的思想去编写类、对象、调用相应的方法等，ORM会将其转换/映射成原生SQL然后交给数据库引擎来执行
- 类对应一张表；属性对应字段；一个类的实例对象对应表中一条数据
- ORM帮我们做了SQL优化且支持多种数据库，这极大地提升了我们的开发效率

Flask-SQLAlchemy

Flask-SQLAlchemy

- Flask-SQLAlchemy是一个Flask扩展，简化了在Flask应用使用SQLAlchemy 的操作
- SQLAlchemy 是一个强大的关系型数据库框架，支持多种数据库后台
- SQLAlchemy 提供了高层 ORM，也提供了使用数据库原生 SQL 的低层功能
- SQLAlchemy 是目前python中最强大的 ORM框架，功能全面，使用简单
- 使用 pip install flask-sqlalchemy 命令进行安装
- 还需要pip install mysqlclient 或 pip install pymysql 安装数据库引擎的依赖包

https://blog.csdn.net/weixin_47906106/article/details/123774620

http://www.pythondoc.com/flask-sqlalchemy/

第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

基本实现

- 实现一个基于Flask-SQLAlchemy的数据操作基本流程
  - 导入与配置
  - 初始化
  - 构建模型类
  - 数据库基本操作

基本实现

导入与配置

- 导入 from flask_sqlalchemy import SQLAlchemy
- 配置
  - ✓ 提前在数据库软件中构建一个空数据库
  - ✓ flask-sqlalchemy 的相关配置也封装到了 flask 的配置项中, 可以通过app.config属性 或 配置加载方案 (如config.from_object) 进行设置
  - ✓ 配置数据库引擎

```
dialect+driver://username:password@host:port/database
mysql://scott:tiger@localhost/mydatabase
oracle://scott:tiger@127.0.0.1:1521/sidname
sqlite:////absolute/path/to/foo.db
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = "mysql+pymysql://root:123456@127.0.0.1:3306/flask_demo“
```

  - ✓ 配置是否需要跟踪对象变化，一般设置为False

```
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
```

基本实现

导入与配置

更多配置

SQLALCHEMY_DATABASE_URI	用于连接数据的数据库。例如: •sqlite:////tmp/test.db •mysql://username:password@server/db
SQLALCHEMY_BINDS	一个映射绑定 (bind) 键到 SQLAlchemy 连接 URIs 的字典。 更多的信息请参阅绑定多个数据库。
SQLALCHEMY_ECHO	如果设置成 *True*，SQLAlchemy 将会记录所有发到标准输出(stderr)的语句，这对调试很有帮助。
SQLALCHEMY_RECORD_QUERIES	可以用于显式地禁用或者启用查询记录。查询记录在调试或者测试模式下自动启用。更多信息请参阅 get_debug_queries()。
SQLALCHEMY_NATIVE_UNICODE	可以用于显式地禁用支持原生的unicode。这是某些数据库适配器必须的（像在 Ubuntu 某些版本上的 PostgreSQL），当使用不合适的指定无编码的数据库 默认值时。
SQLALCHEMY_POOL_SIZE	数据库连接池的大小。默认是数据库引擎的默认值 （通常是 5）。
SQLALCHEMY_POOL_TIMEOUT	指定数据库连接池的超时时间。默认是 10。
SQLALCHEMY_POOL_RECYCLE	自动回收连接的秒数。这对 MySQL 是必须的，默认情况下 MySQL 会自动移除闲置 8 小时或者以上的连接。 需要注意地是如果使用 MySQL 的话， Flask-SQLAlchemy 会自动地设置这个值为 2 小时。
SQLALCHEMY_MAX_OVERFLOW	控制在连接池达到最大值后可以创建的连接数。当这些额外的 连接回收到连接池后将会被断开和抛弃。
SQLALCHEMY_TRACK_MODIFICATIONS	如果设置成 *True* (默认情况)，Flask-SQLAlchemy 将会追踪对象的修改并且发送信号。这需要额外的内存， 如果不必要的可以禁用它。

基本实现

初始化

- 方式一：初始化组件对象，直接关联Flask应用，需提前初始化app

  db = SQLAlchemy(app)

- 方式二：初始化组件对象，延后关联Flask应用

  db = SQLAlchemy()　　提前初始化数据库组件对象

  db.init_app(app)　　在工厂函数中关联Flask应用

```python
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

# 应用配置
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql://root:mysql@127.0.0.1:3306/test31'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SQLALCHEMY_ECHO'] = True

# 方式1：初始化组件对象，直接关联flask应用
db = SQLAlchemy(app)
```

```python
from flask import Flask
from flask_sqlalchemy import SQLAlchemy


# 方式2：初始化组件对象，延后关联flask应用
db = SQLAlchemy()


def create_app(config_type):
    """工厂函数"""

    # 创建应用
    flask_app = Flask(__name__)
    # 加载配置
    config_class = config_dict[config_type]
    flask_app.config.from_object(config_class)

    # 关联flask应用
    db.init_app(app)

    return flask_app
```

基本实现

构建模型类

- 模型类必须继承 db.Model，其中 db 指对应的组件对象
- 表名默认为类名小写，可以通过 __tablename__类属性进行修改
- 类属性对应字段，必须是通过 db.Column() 创建的对象
- 可以通过 create_all() 和 drop_all()方法来创建和删除所有模型类对应的表

```
class Role(db.Model):
    __tablename__ = 'role'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(64),unique=True)
    def __repr__(self):
        return '<Role %r>' % self.name
```

```
class User(db.Model):
    __tablename__ = 'user'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True)
    def __repr__(self):
        return '<User %r>' % self.username
```

基本实现

构建模型类

最常用的SQLAlchemy列类型

类型名	Python类型	说明
Integer	int	普通整数，通常是 32 位
SmallInteger	int	取值范围小的整数，通常是 16 位
BigInteger	int 或 long	不限制精度的整数
Float	float	浮点数
Numeric	decimal.Decimal	定点数
String	str	变长字符串
Text	str	变长字符串，对较长或不限长度的字符串做了优化
Unicode	unicode	变长 Unicode 字符串
UnicodeText	unicode	变长 Unicode 字符串，对较长或不限长度的字符串做了优化
Boolean	bool	布尔值
Date	datetime.date	日期
Time	datetime.time	时间
DateTime	datetime.datetime	日期和时间
Interval	datetime.timedelta	时间间隔
Enum	str	一组字符串
PickleType	任何 Python 对象	自动使用 Pickle 序列化
LargeBinary	str	二进制 blob

最常用的SQLAlchemy列选项

选项名	说明
primary_key	如果设为 True，列为表的主键
unique	如果设为 True，列不允许出现重复的值
index	如果设为 True，为列创建索引，提升查询效率
nullable	如果设为 True，列允许使用空值；如果设为 False，列不允许使用空值
default	为列定义默认值

基本实现

数据库基本操作

- 需在with app.app_context():语句下执行数据库基本操作
- 创建表 db.create_all()； 一般在新建表前删除所有的表 db.drop_all()
- 插入行
  - ✓ 新建表格类的实例对象 role_admin = Role(name = 'Admin')
  - ✓ 将对象添加进数据库会话 db.session.add(role_admin)
  - ✓ 执行会话提交 db.session.commit() 若出错可回滚 db.session.rollback()
- 修改行
  - ✓ 类实例中修改相关属性 role_admin.name = 'Administrator‘
  - ✓ 将修改后的对象添加进数据库会话 db.session.add(role_admin)
  - ✓ 执行会话提交 db.session.commit()
- 删除行 会话中删除实例 db.session.delete(role_mod) 执行会话提交 db.session.commit()
- 查询行 使用query对象 Role.query.all()

第一次 实验课安排

成都信息工程大學
Chengdu University of Information Technology

实验一：Web应用开发基础实训项目

2023年11月20日 9-12节(18:30) B511

- 要求：
  - 独立完成实验、实验报告撰写
  - 12月1日前提交至　　yyg@cuit.edu.cn
  - 电子稿，提交附加的文件名命名为：学号+姓名+Web应用开发C实验1，

    如：　“2022046046李玲Web应用开发C实验1”

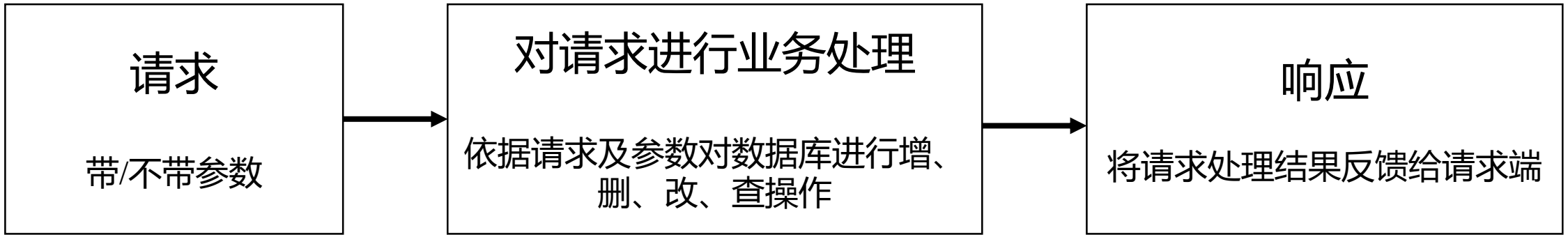
第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

在视图函数中操作数据库

数据库操作

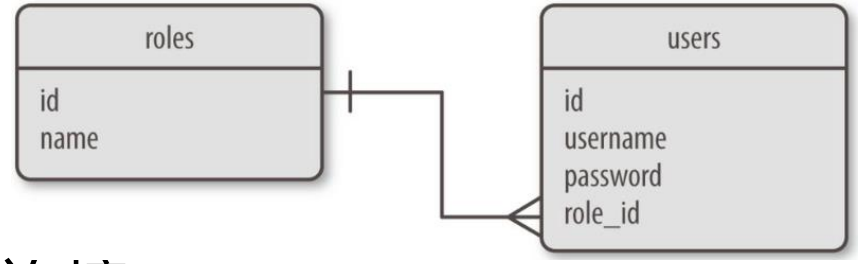
- 前面我们提到的数据库操作都是放在主函数中，这样是为了方便演示
- 一般实际应用中都将对数据库操作，即数据库的业务逻辑放到视图函数中来进行，这样一个视图函数就能实现一个特定的功能，实现系统的模块化管理，进而实现“高内聚、低耦合”



第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

关系构建



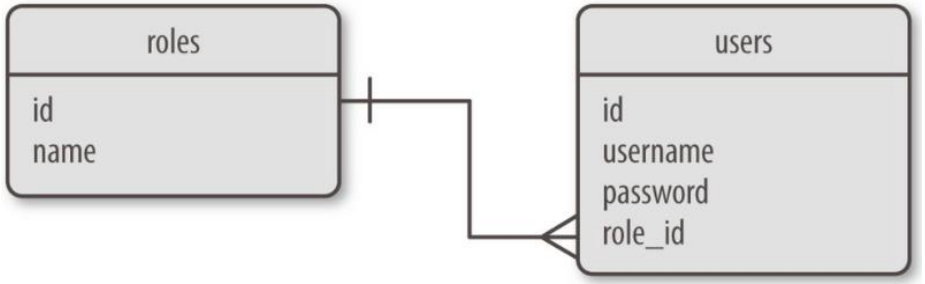
一对多

- 关系型数据库使用关系把不同表中的行联系起来
- 在多端创建一个外键属性，能将其与一端的id建立连接 role_id = db.Column(db.Integer, db.ForeignKey('role.id'))
- 在一端用relationship定义一个多端的列表，用于查询一端中有多少多端的实例；这种作用主要体现在View和Template中，不会体现在数据库内 users = db.relationship('User', backref='role')

选项名	说明
backref	在关系的另一个模型中添加反向引用
primaryjoin	明确指定两个模型之间使用的联结条件；只在模棱两可的关系中需要指定
lazy	指定如何加载相关记录，可选值有 select （首次访问时按需加载）、immediate （源对象加载后就加载）、joined （加载记录，但使用联结）、subquery （立即加载，但使用子查询），noload （永不加载）和 dynamic （不加载记录，但提供加载记录的查询）
uselist	如果设为 False，不使用列表，而使用标量值
order_by	指定关系中记录的排序方式
secondary	指定多对多关系中关联表的名称

关系构建

一对多




```
class Role(db.Model):
    __tablename__ = 'role'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(64), unique=True)
    users = db.relationship('User', backref='role')

    def __repr__(self):
        return '<Role %r>' % self.name
```

```
class User(db.Model):
    __tablename__ = 'user'
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True)
    role_id = db.Column(db.Integer, db.ForeignKey('role.id'))

    def __repr__(self):
        return '<User %r>' % self.username
```

```
@app.route('/user')
def userDetails():
    users =  User.query.filter_by(role_id= 1)
    return render_template('UserDetail.html',users=users)
```

```
{% for user in users %}
    {{user.username}} <br>
    {{user.role.name}} <br>
{% endfor %}
```

关系构建

一对一

- 一对一的关系可以在一对多的基础上进行表示，只是在调用db.relationship()时要把uselist设置为False，将其中的多变为1即可

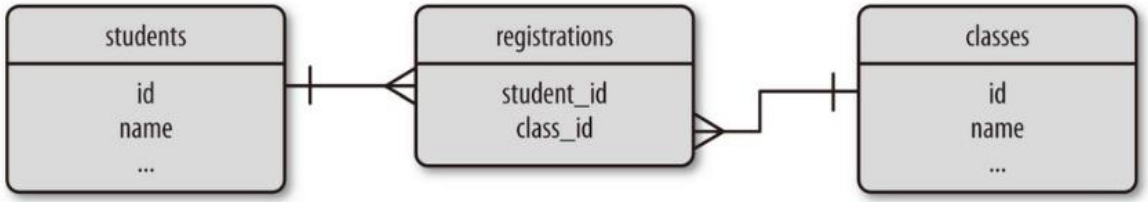
```
class Male(db.Model):
    __tablename__ = 'male'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(20),unique=True)
    wife = db.relationship('Female', backref='husband', uselist=False)
```

```
class Female(db.Model):
    __tablename__ = 'female'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(20),unique=True)
    husband_id = db.Column(db.Integer,db.ForeignKey('male.id'))
```

多对一

- 也可以使用一对多来表示，对调两个表表即可，或者把外键和db.relationship()都写在多的这一侧

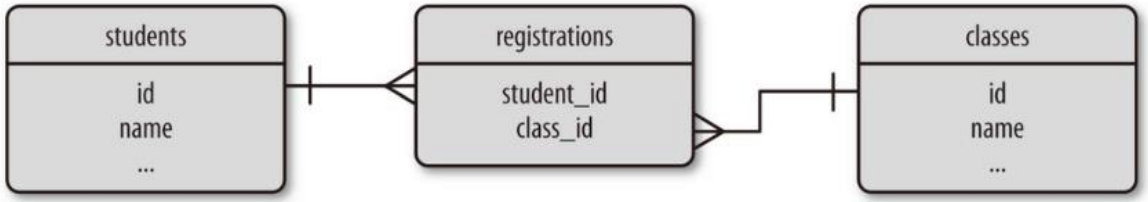
关系构建



- 多对多
  - 使用第三张表完成多对多的关联关系
  - 将多对多的关系分解为原表和关联表之间的两个一对多的关系
  - 多对多关系仍使用定义一对多关系的db. relationship()方法定义，但在多对多关系中，必须把secondary参数设为关联表
  - 多对多关系可以在任何一个类中定义，backref 参数会处理好关系的另一侧
  - 关联表就是一个简单的表，不是模型，SQL Alchemy会自动接管这个表

关系构建

多对多




```
registrations = db.Table('registrations',
    db.Column('student_id', db.Integer, db.ForeignKey('students.id')),
    db.Column('class_id', db.Integer, db.ForeignKey('classes.id'))
)
```

```
class Student(db.Model):
    __tablename__ = 'students'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(20))
    classes = db.relationship('Class',
                              secondary=registrations,
                              backref=db.backref('students', lazy='dynamic'),
                              lazy='dynamic')
```

```
class Class(db.Model):
    __tablename__ = 'classes'
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(20))
```

```
{% for student in studentes %}
    {{student.name}} <br>
    {% for class in student.classes %}
        {{class.name}}<br>
    {% endfor %}
{% endfor %}
```

第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

数据库操作扩展

- 查询对象
  - Flask-SQLAlchemy为每个模型类都提供了query对象　　模型类.query
  - 查询所有：模型类.query.all()　　User.query.all()　　得到所有结果的列表
- 过滤器查询 (有条件查询)
  - filter_by()　　模型类.query.filter_by(字段名 = 值)　　得到符合条件的列表
    - ✓ all()　　模型类.query.filter_by().all()　　得到符合条件的列表的所有结果
    - ✓ first()　　模型类.query.filter_by().first()　　得到符合条件的第一个对象
  - get()　　模型类.query.get(1)　　根据主键查询内容,返回一个对象
  - filter()　　模型类.query.filter(模型类.字段==值)　　得到符合条件的列表
    - ✓ all()　　模型类.query.filter().all()　　得到符合条件的列表的所有结果
    - ✓ first()　　模型类.query.filter().first()　　得到符合条件的第一个 对象
    - ✓ 模糊查询　　模型类.query.filter(模型类.字段.endswith('z')/startswith('z')/contains('z')/like('z%'))
      .all()　　表示查询某字段以字符结束、开始、包含等的所有记录

数据库操作扩展

过滤器查询 (有条件查询)

- filter()　　模型类.query.filter(模型类.字段==值)　　得到符合条件的列表
  - ✓多条件或查询 模型类.query.filter(or_(条件1，条件2…)).all()　　sqlalchemy 中导入or_
  - ✓多条件且查询 模型类.query.filter(and_(条件1，条件2…)).all()　　sqlalchemy 中导入and_
  - ✓非查询　　模型类.query.filter(not_(条件1)).all()　　sqlalchemy 中导入not_
  - ✓对于能比较大小的字段可以使用 <、>、__lt__、__gt__ __le__、__ge__等来表达数据范围
  - ✓列表内查询　　模型类.query.filter(类模型.字段.in_([列表])).all()　　查询该字段列表中的记录
  - ✓数字范围查询 模型类.query.filter(类模型.字段.between(小值,大值)).all()查询某数值范围内的记录
- order_by()　　对查询结果依据某字段来排序　　模型类.query.filter()order_by(字段)
  - ✓ 对表中所有记录排序　模型类.query.order_by(字段)　　默认升序
  - ✓ 实现逆序　　模型类.query.filter().order_by(-模型类.字段)
  - ✓ order_by 的参数分两类：’字段’不能实现倒序；模型类.字段，可在其前加-实现倒序

数据库操作扩展

- 过滤器查询 (有条件查询)
  - limit()　　获取限定数量的记录
    - ✓ 取查询结果中限定数量的结果　　模型类.query.filter().limit(数量)
    - ✓ 若取的值是排序后的，则将limit()放在order_by()之后
    - ✓结合offset()表示偏移一定数量后取几条记录出来，模型类.query.filter().offset(数量).limit(数量)

第八章 Flask-Web数据库

- 数据库
- Flask-SQLAlchemy
- 基本实现
- 在视图函数中操作数据库
- 关系建模
- 数据库操作扩展
- 数据库迁移

数据库迁移

- 迁移的必要性
  - 在应用开发的过程中，不可避免会修改数据库模型，就需要同步更新数据库表
  - 但是flask-SQLAlchemy，只有在数据库中表不存在时，才会依据模型创建新表，如此为了把我修改后的模型进行更新的方式就只有先删除数据库中原有的表，然后再新建表
  - 这样就会丢失原数据表中的数据，如何才能又能保存原数，又能更新模型，最好的方式就是就是是用数据库迁移框架来实现
- Flask-Migrate https://flask-migrate.readthedocs.io/en/latest/
  - 数据库迁移框架可跟踪数据库模式的变化，并以增量的方式将变化更新至数据库
  - Flask-Migrate是对SQLAlchemy开发人员所编写的迁移框架Alembic的轻量级包装，并与flask命令做了集成
  - 安装：pip install flask-migrate

数据库迁移

迁移步骤

创建迁移仓库

- ✓ 项目启动文件中导入：from flask_migrate import Migrate
- ✓ 项目启动文件中实例化：migrate = Migrate(app,db)
- ✓ 终端命令行中 执行：flask db init
- ✓ 在项目的目录下新建migrations目录，目录中包含了与数据库迁移需要的相关信息

创建迁移脚本

- ✓ 终端命令行中执行： flask db migrate [–m “修改说明”]，在version文件夹下生成迁移脚本xxx.py

更新数据

- ✓ 终端命令行执行： flask db upgrade，实现模型改动向数据库的迁移
- ✓ 终端命令行执行： flask db downgrade，依据迁移文件回退到数据库上一次修改的地方

第八章 Flask-Web数据库 总结

数据库	基本概念
Flask-SQLAlchemy	ORM、简介
基本实现	导入配置、初始化、构建模型类、数据库基本操作
在视图函数中操作数据库	数据库操作
关系建模	一对多、多对对
数据库操作扩展	多种查询方式
数据库迁移	flask-migrate的使用